

K-Means Clustering

Lab 1

Objective:

To implement the k-means clustering algorithm on a randomly generated dataset using Python.

Materials:

- Python installed on your computer
- Libraries: NumPy, matplotlib, scikit-learn

Steps:

1. Setup and Data Generation

- Import necessary libraries:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

- Set a random seed and create a random dataset:

```
np.random.seed(42)
X = np.random.rand(300, 2)
```

2. Visualise the Dataset

- Plot the generated data points:

```
plt.scatter(X[:, 0], X[:, 1], s=50)

plt.xlabel('Feature X')
plt.ylabel('Feature Y')
plt.title('Scatter Plot of Random Data')

plt.show()
```

3. Apply K-means Clustering

- Initialise the KMeans object and fit it to the data:

```
kmeans = KMeans(n_clusters=3)
kmeans.fit(X)
```

- Retrieve the cluster centres and labels:

```
centers = kmeans.cluster_centers_
labels = kmeans.labels_
```

4. Visualise the Clusters

- Plot the data points with cluster assignments and centroids:

```
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75)
plt.show()
```

Explanation:

- **Step 1:** We import necessary libraries and generate a random dataset with 300 points in 2D space.
- **Step 2:** We visualise the randomly generated data to understand its distribution.
- **Step 3:** We apply the k-means clustering algorithm to group the data into three clusters.
- **Step 4:** We visualise the resulting clusters and their centroids.

Conclusion:

By completing this lab, you will understand how to implement and visualise k-means clustering using Python. You will also learn how to interpret the clustering results and the significance of centroids in the clustering process.

Sources

1. gist.github.com - Lab on K-means clustering
2. ocw.cs.pub.ro - Lab 9. Unsupervised Learning. K-Means Clustering
3. domino.ai - Getting started with k-means clustering in Python
4. datacamp.com - Introduction to k-Means Clustering with scikit-learn in Python
5. realpython.com - K-Means Clustering in Python: A Practical Guide
6. <https://www.kaggle.com/code/satishgunjal/tutorial-k-means-clustering>

Lab 2

Objective

Implement the K-means clustering algorithm on a dataset and use the elbow method to determine the optimal number of clusters.

Materials

- Python environment
- Libraries: NumPy, matplotlib, sklearn

Steps

1. Import Libraries

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

2. Generate or Load Data

Generate a random dataset for this lab. Alternatively, you can load a dataset from a file or a library.

```
np.random.seed(42)
X = np.random.rand(300, 2)
```

3. Visualise the Dataset

```
plt.scatter(X[:, 0], X[:, 1], s=50)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Scatter Plot of Random Data')
plt.show()
```

4. Apply K-Means Clustering with Multiple Values of K

Perform K-means clustering with different values of K and compute the Within-Cluster Sum of Squares (WCSS) for each K.

```
wcss = []
for k in range(1, 11):
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

# Plot the WCSS to use the Elbow Method
plt.plot(range(1, 11), wcss, marker='o')
plt.title('Elbow Method for Optimal K')
```

```
plt.xlabel('Number of Clusters')
plt.ylabel('WCSS')
plt.show()
```

5. Determine the Optimal Number of Clusters

Identify the elbow point in the plot to determine the optimal number of clusters.

- The elbow point is where the WCSS starts to decrease more slowly, forming an "elbow."

6. Fit K-Means with Optimal K

From the elbow plot, select the optimal K (e.g., 3 in this example) and fit the K-means model.

```
optimal_k = 3
kmeans = KMeans(n_clusters=optimal_k, random_state=42)
kmeans.fit(X)

centers = kmeans.cluster_centers_
labels = kmeans.labels_

# Plot the clustered data
plt.scatter(X[:, 0], X[:, 1], c=labels, s=50, cmap='viridis')
plt.scatter(centers[:, 0], centers[:, 1], c='red', s=200, alpha=0.75,
            marker='X')
plt.title(f'K-Means Clustering with {optimal_k} Clusters')
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.show()
```

Explanation

- **Import Libraries:** Necessary for data manipulation and visualisation.
- **Generate or Load Data:** Creates a sample dataset with random points.
- **Visualise the Dataset:** Helps understand data distribution.
- **Apply K-Means Clustering:** Calculates WCSS for different K values.
- **Determine the Optimal Number of Clusters:** Uses the elbow method to find the optimal K.
- **Fit K-Means with Optimal K:** Fits the model with the optimal number of clusters and visualises the results.

Conclusion

By following this lab, you will learn how to implement K-means clustering, determine the optimal number of clusters using the elbow method, and visualise the results.

Sources

1. [GeeksforGeeks - Elbow Method for optimal value of k in KMeans.](#)
2. [Analytics Vidhya - Elbow Method for Optimal Cluster Number in K-Means](#)
3. [Wikipedia - Elbow method \(clustering\)](#)
4. [BuiltIn - Stop Using Elbow Method in K-means Clustering](#)
5. [Medium - What is the purpose of the elbow method in K-means](#)
6. [OpenAI Blog - Optimizing K-Means Clustering: A Guide to Using the Elbow](#)